# Guia de Estudo: Model Context Protocol (MCP) e GitHub CLI

Este guia de estudo foi elaborado para auxiliar na revisão e fixação dos conceitos fundamentais referentes ao **Model Context Protocol (MCP)** e à **GitHub CLI ( gh )**, cobrindo arquitetura, especificações técnicas, implementação prática e comandos de linha de comando.

## Questionário de Avaliação (10 Perguntas de Resposta Curta)

1. Qual é a principal finalidade do Model Context Protocol (MCP) e em qual protocolo do ecossistema de desenvolvimento ele se inspira?
2. Quais são os três papéis principais estabelecidos pela arquitetura do MCP para a troca de mensagens?
3. Quais são os três recursos centrais que um servidor MCP pode oferecer a um cliente?
4. Como é estruturada a comunicação de transporte no exemplo prático de criação de um servidor MCP em TypeScript para o Claude Code?
5. De acordo com a especificação do MCP, quais são os três princípios fundamentais de segurança e privacidade que os desenvolvedores devem seguir?
6. O que é a GitHub CLI (gh) e qual é a sua principal vantagem em relação ao uso do navegador web tradicional?
7. Qual é a estrutura sintática padrão que a maioria dos comandos do GitHub CLI segue?
8. Como pode ser feita a instalação da GitHub CLI nos sistemas operacionais Windows, macOS e Linux (Debian/Ubuntu)?
9. De que forma a combinação da flag --json com a ferramenta jq auxilia no uso da GitHub CLI?
10. Qual é a diferença entre a GitHub CLI e o Git Bash?

## Chave de Respostas

1. O Model Context Protocol (MCP) é um protocolo aberto projetado para padronizar a integração entre aplicações de Modelos de Linguagem (LLM) e fontes de dados ou ferramentas externas. O MCP inspira-se no *Language Server Protocol* (LSP), que padronizou a integração de linguagens de programação em ferramentas de desenvolvimento.
2. A arquitetura do MCP define os papéis de **Hosts** (aplicações de LLM que iniciam as conexões), **Clients** (conectores internos da aplicação host) e **Servers** (serviços que fornecem contexto, dados e capacidades para o sistema).
3. Os servidores MCP podem oferecer **Resources** (dados e contextos para o usuário ou modelo), **Prompts** (mensagens e fluxos de trabalho modelados) e **Tools** (funções executáveis pelo modelo de IA).
4. No exemplo prático em TypeScript, a comunicação é estabelecida via entrada e saída padrão ( *Standard IO* ) utilizando a classe stdioServerTransport. O servidor é então registrado no Claude Code por meio do comando claude mcp add, permitindo a execução de scripts e ferramentas locais.
5. Os três princípios fundamentais de segurança são **Consentimento e Controle do Usuário** (autorização explícita para ações e dados), **Privacidade de Dados**

(proteção e não transmissão não autorizada) e **Segurança de Ferramentas** (tratamento de execuções de código como potencialmente perigosas).

6. A GitHub CLI (gh) é uma ferramenta de linha de comando que permite interagir diretamente com o GitHub a partir do terminal. Sua principal vantagem é eliminar a necessidade de alternar o contexto entre o ambiente de desenvolvimento e as abas do navegador web ao gerenciar repositórios, pull requests e issues.
7. A sintaxe padrão segue a estrutura gh command subcommand flags/options. Onde command indica a categoria (ex: repo, issue, pr), subcommand define a ação específica (ex: create, list) e as flags/options refinam o comportamento do comando.
8. No Windows, a instalação pode ser realizada via Winget com winget install --id GitHub.cli (ou instalador .msi); no macOS, utiliza-se o Homebrew com brew install gh; e no Linux Debian/Ubuntu, utiliza-se o gerenciador de pacotes via sudo apt install gh.
9. A flag --json permite exportar dados estruturados de comandos da gh selecionando apenas os campos desejados. Em conjunto com o jq, é possível formatar esses dados no terminal para construir scripts de automação, pipelines de CI/CD e changelogs automáticos.
10. O Git Bash é um aplicativo de terminal para Windows que provê um ambiente Bash para rodar comandos do Git. Já a GitHub CLI é uma ferramenta distinta que adiciona comandos específicos do GitHub (como gerenciamento de PRs e issues) ao terminal, funcionando em conjunto com o Git.

## Questões Dissertativas

1. Discuta como a arquitetura do Model Context Protocol (MCP) e suas extensões opcionais (como *Tasks* , *Skills over MCP* e *MCP Apps* ) permitem expandir o ecossistema de aplicações de IA, mantendo a modularidade e a interoperabilidade.
2. Analise os desafios e as diretrizes de segurança na implementação do MCP, explicando detalhadamente o motivo pelo qual as descrições e invocações de ferramentas ( *Tools* ) devem ser tratadas como código não confiável.
3. Elabore um plano de automação de fluxo de trabalho para uma equipe de desenvolvimento utilizando a GitHub CLI (gh), abordando a criação de *aliases* personalizados, triagem de *issues* e monitoramento de *workflows* do GitHub Actions diretamente pelo terminal.
4. Descreva o processo passo a passo para desenvolver e registrar um servidor MCP local utilizando TypeScript e a biblioteca Zod, explicando o papel do transporte via *Standard IO* e da ferramenta npx/TSX na integração com o Claude Code.
5. Compare os recursos de gerenciamento de repositórios e de Pull Requests oferecidos pela interface gráfica web do GitHub com as capacidades interativas e não interativas (via flags) disponibilizadas pela GitHub CLI.

## Glossário de Termos-Chave

Termo,Definição Baseada nas Fontes
Model Context Protocol (MCP),Protocolo aberto baseado em JSON-RPC 2.0 que padroniza a conexão entre aplicações de IA/LLM e ferramentas ou fontes de dados externas.
Host (MCP),Aplicação baseada em LLM que inicia e mantém conexões com servidores MCP.

Client (MCP),Conector interno localizado dentro da aplicação Host que gerencia a comunicação com os servidores.
Server (MCP),"Serviço ou aplicação que expõe contexto, dados, rotinas ou ferramentas para uso pelas aplicações de IA através do protocolo MCP."
JSON-RPC 2.0,Formato de troca de mensagens leves e autônomas ( stateless ) utilizado como protocolo base para a comunicação do MCP.
Language Server Protocol (LSP),"Protocolo padrão para integração de linguagens de programação em IDEs, que serviu de inspiração para a concepção do MCP."
Resources (MCP),Funcionalidade do servidor MCP que fornece dados e contextos reutilizáveis para a aplicação de IA ou para o usuário.
Prompts (MCP),Modelos pré-definidos de mensagens e fluxos de trabalho disponibilizados pelos servidores MCP aos usuários.
Tools (MCP),Funções executáveis disponibilizadas por servidores MCP que representam caminhos de execução de código arbitrário pelo modelo de IA.
Elicitation (MCP),Recurso iniciado pelo servidor MCP para solicitar informações adicionais diretamente ao usuário através do cliente.
Standard IO (stdio),Mecanismo de transporte de entrada/saída padrão utilizado para conectar localmente um servidor MCP a um cliente/host como o Claude Code.
Zod,"Biblioteca em TypeScript utilizada para definição e validação de esquemas de dados, empregada na validação de argumentos de ferramentas no MCP."
Claude Code,Ferramenta ou ambiente local configurável que permite registrar e executar servidores e ferramentas MCP locais.
GitHub CLI ( gh ),"Ferramenta de linha de comando oficial do GitHub para execução de tarefas de gerenciamento de código, issues e PRs sem sair do terminal."
Git Bash,Aplicação de terminal para sistemas Windows que provê um ambiente Unix/Bash para execução de comandos Git.
Pull Request (PR),"Mecanismo do GitHub para propor e revisar alterações de código, gerenciável na CLI através do grupo de comandos gh pr."
GitHub Actions,"Plataforma de automação e CI/CD do GitHub, cujos workflows e runs podem ser monitorados e reiniciados via gh run."
Alias (GitHub CLI),Atalho de comando customizado criado na GitHub CLI (gh alias set) para simplificar sequências de comandos frequentes.
jq,"Utilitário de linha de comando para processamento e formatação de dados JSON, comumente usado em conjunto com a flag --json da gh."
Winget / Homebrew,"Gerenciadores de pacotes utilizados para instalação simplificada da GitHub CLI nos sistemas Windows e macOS, respectivamente."